GUIA TÉCNICO

Redes recorrentes, LSTM e GRU

Capítulo 18/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Explicar estados ocultos, dependências temporais e controle por portas.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

RNNs processam sequências com estado oculto compartilhado. O gradiente pode desaparecer/explodir em sequências longas; LSTM e GRU usam portas para reter e atualizar informação. Shapes e máscaras distinguem sequências de comprimentos variáveis.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: h_t=tanh(W_x x_t+W_h h_(t−1)+b)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Estados recorrentes, gradientes no tempo e portas de memória ajudam a explicar RNN/LSTM/GRU e suas limitações.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(0)
x=np.array([.2,.3,.4]);h=0.;wx=.8;wh=.35
estados=[]
for t in x:
 h=np.tanh(wx*t+wh*h);estados.append(h)
print('Estados da RNN escalar:',np.round(estados,4).tolist())
assert len(estados)==3
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados

reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Sensibilidade a estados iniciais em RNN escalar

Estados recorrentes, gradientes no tempo e portas de memória ajudam a explicar RNN/LSTM/GRU e suas limitações.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
seq=np.ones(25)*.1
def rnn(h0,wh):
 h=h0
 for x in seq:h=np.tanh(.2*x+wh*h)
 return h
h=1e-5
for wh in [.2,.6,.95]:
 derivative=(rnn(h,wh)-rnn(-h,wh))/(2*h)
 print('peso recorrente',wh,'sensibilidade ao estado inicial',round(derivative,8))
assert abs(rnn(h,.2)-rnn(-h,.2))<.1
```

É um experimento ilustrativo, não diagnóstico universal de gradientes: saturação de tanh e pesos recorrentes podem alterar o comportamento.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que LSTM pode superar RNN simples em sequências longas?

Resposta: Suas portas e caminho de memória ajudam a preservar informação e reduzir o desaparecimento de gradientes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide

REVISÃO DIDÁTICA · CAPÍTULO 18

RNN, LSTM e GRU: estados e portas

1. Conceito, escopo e interpretação

Uma RNN compartilha parâmetros ao longo do tempo e atualiza estado oculto. LSTM tem estado de célula c_t e portas de esquecimento, entrada e saída. GRU combina atualização e reinicialização em estrutura mais compacta.

2. Formalização e exemplo resolvido

RNN: h_t=tanh(W_x x_t+W_h h_(t-1)+b). LSTM: f_t=sigma(W_f[x_t,h_prev]+b_f); i_t=sigma(...); g_t=tanh(...); c_t=f_t*c_prev+i_t*g_t; o_t=sigma(...); h_t=o_t*tanh(c_t). GRU: z_t=sigma(...); r_t=sigma(...); h_t=(1-z_t)*n_t+z_t*h_prev (uma convenção comum).

3. Relação com o notebook e experimento

No notebook, use a RNN escalar para estudar sensibilidade a estados iniciais e gradientes ao longo da sequência. Para testar LSTM/GRU de fato, construa camadas keras.layers.LSTM e GRU em laboratório opcional e compare mesma tarefa e orçamento.

4. Limitações e erros frequentes

A demonstração numérica da RNN simples não implementa portas LSTM nem GRU. A interpretação exata de z_t varia conforme convenção adotada; declare a definição ao apresentar equações.

5. Exercício comentado

Se f_t aproxima 1 e i_t aproxima 0, o que ocorre com c_t na LSTM? O estado anterior tende a ser preservado. Por que isso pode ajudar gradientes? Oferece caminho de memória mais estável.

6. Ampliação conceitual do guia

A RNN simples atualiza h_t=tanh(W_x*x_t+W_h*h_(t-1)+b). Ao longo de muitos passos, o produto sucessivo de derivadas pode reduzir ou amplificar gradientes. LSTM usa estado de célula e portas de entrada, esquecimento e saída; GRU usa portas de atualização e reset.

7. Orientação de estudo e validação

O notebook mede sensibilidade do estado final a mudanças no estado inicial em uma RNN escalar. Esse experimento ajuda a estudar efeitos do peso recorrente e saturação, mas não é uma implementação de LSTM ou GRU nem uma prova universal sobre gradientes. Explique máscaras e padding ao processar sequências variáveis.

8. Exercício adicional de fixação

Exercício: por que uma GRU pode ter menos parâmetros que uma LSTM de dimensão equivalente? Resposta: costuma usar menos conjuntos de portas, embora a contagem dependa da implementação.

Verificação: execute o notebook homônimo no ambiente apropriado; registre versões, shapes, sementes e métricas. Laboratórios Keras opcionais não foram executados durante esta revisão documental.

Abrir notebook do capítulo no repositório

LABORATÓRIO COMPLEMENTAR · CAPÍTULO 18

Equações completas das portas — LSTM e GRU

Implementação ilustrativa / formalização

```
LSTM (uma convenção comum):
f_t = sigmoid(W_f [h_(t-1), x_t] + b_f)
i_t = sigmoid(W_i [h_(t-1), x_t] + b_i)
g_t = tanh   (W_g [h_(t-1), x_t] + b_g)
o_t = sigmoid(W_o [h_(t-1), x_t] + b_o)
c_t = f_t * c_(t-1) + i_t * g_t
h_t = o_t * tanh(c_t)

GRU (convenção com z_t como porta de retenção):
z_t = sigmoid(W_z [h_(t-1), x_t] + b_z)
r_t = sigmoid(W_r [h_(t-1), x_t] + b_r)
n_t = tanh(W_n [r_t*h_(t-1), x_t] + b_n)
h_t = z_t*h_(t-1) + (1-z_t)*n_t
```

Interpretação e limites

O símbolo * representa multiplicação elemento a elemento. LSTM mantém c_t separado de h_t; GRU não mantém estado de célula independente. Algumas fontes definem a porta de atualização de forma complementar: compare a convenção antes de confrontar fórmulas.

Como verificar

Compare as equações com o notebook e confira a convenção de cada porta. Esta página apresenta uma formalização matemática, não código executável.

Referência: notebook do mesmo capítulo em 07_deep_learning (GitHub).